

primitive	signature	Verilog translation & example output	CVC5 translation	comment
uadd_x_y	UInt(x),UInt(y) → UInt(max(x,y)+1)	<pre> if name.startswith('uadd_'): x, y = map(int, name[len('uadd_'):].split('_')) out = max(x, y) + 1 le, lw, next_idx = convert(next_idx) re, rw, next_idx = convert(next_idx) l = _zext(le, lw, out, emit_forced) r = _zext(re, rw, out, emit_forced) t, tw = emit(out, f"{l} + {r}") return t, tw, next_idx //uadd_8_6 wire [5:0] t0 = 6'd0; wire [8:0] t1 = {{1'b0}}, ma} + {{3'1'b0}}, t0}; </pre>	<pre> if name.startswith('uadd_'): x, y = map(int, name[len('uadd_'):].split('_')) out = max(x, y) + 1 left, next_idx = convert(next_idx) right, next_idx = convert(next_idx) l = _zext(solver, left, out) r = _zext(solver, right, out) return solver.mkTerm(Kind.BITVECTOR_ADD, l, r), next_idx </pre>	condition: $\max(x,y) \leq 13$; x,y ∈ {1..13}; 169 pairs total
usub_x_y	UInt(x),UInt(y) → UInt(max(x,y)+1)	<pre> if name.startswith('usub_'): x, y = map(int, name[len('usub_'):].split('_')) out = max(x, y) + 1 le, lw, next_idx = convert(next_idx) re, rw, next_idx = convert(next_idx) l = _zext(le, lw, out, emit_forced) r = _zext(re, rw, out, emit_forced) t, tw = emit(out, f"({l} >= {r}) ? ({l} - {r}) : {out}'d0") return t, tw, next_idx //usub_8_6 wire [5:0] t0 = 6'd0; wire [8:0] t1 = (({{1'b0}}, ma} >= {{3'1'b0}}, t0)) ? ({{1'b0}}, ma} - {{3'1'b0}}, t0) : 9'd0; </pre>	<pre> if name.startswith('usub_'): x, y = map(int, name[len('usub_'):].split('_')) out = max(x, y) + 1 left, next_idx = convert(next_idx) right, next_idx = convert(next_idx) return _usub_bv(solver, left, right, out), next_idx </pre>	condition: $\max(x,y) \leq 13$; x,y ∈ {1..13}; 169 pairs total
uhigh_x_y	UInt(x) → UInt(y) (y < x)	<pre> if name.startswith('uhigh_'): src, dst = map(int, name[len('uhigh_'):].split('_')) shift = src - dst ce, cw, next_idx = convert(next_idx) shifted, _ = emit_forced(src, f"{ce} >> {shift}") t, tw = emit(dst, f"{shifted}[{dst - 1}:0]") return t, tw, next_idx //uhigh_8_4 wire [7:0] t0 = ma >> 4; wire [3:0] t1 = t0[3:0]; </pre>	<pre> if name.startswith('uhigh_'): src, dst = map(int, name[len('uhigh_'):].split('_')) child, next_idx = convert(next_idx) shift_amount = solver.mkBitVector(src, src - dst) shifted = solver.mkTerm(Kind.BITVECTOR_LSHR, child, shift_amount) return solver.mkTerm(solver.mkOp(Kind.BITVECTOR_EXTRACT, dst - 1, 0), shifted), next_idx </pre>	take the highest dst_bits of the src_bits condition: $y < x$; $x \in \{2..14\}$; 91 pairs total
ulow_x_y	UInt(x) → UInt(y) (y < x)	<pre> if name.startswith('ulow_'): src, dst = map(int, name[len('ulow_'):].split('_')) ce, cw, next_idx = convert(next_idx) ce, _ = emit_forced(cw, ce) t, tw = emit(dst, f"{ce}[{dst - 1}:0]") return t, tw, next_idx //ulow_8_4 wire [7:0] t0 = ma; wire [3:0] t1 = t0[3:0]; </pre>	<pre> if name.startswith('ulow_'): src, dst = map(int, name[len('ulow_'):].split('_')) child, next_idx = convert(next_idx) return solver.mkTerm(solver.mkOp(Kind.BITVECTOR_EXTRACT, dst - 1, 0), child), next_idx </pre>	take the lowest dst_bits of the src_bits condition: $y < x$; $x \in \{2..14\}$; 91 pairs total
ucast_x_y	UInt(x) → UInt(y) (x ≠ y)	<pre> if name.startswith('ucast_'): x, y = map(int, name[len('ucast_'):].split('_')) ce, cw, next_idx = convert(next_idx) if y >= x: t, tw = emit(y, _zext(ce, cw, y, emit_forced)) return t, tw, next_idx t, tw = _clip_unsigned_verilog(emit, emit_forced, ce, cw, y) return t, tw, next_idx //ucast_8_5 wire [7:0] t0 = ((ma > 8'd31)) ? 8'd31 : ma; wire [4:0] t1 = t0[4:0]; </pre>	<pre> if name.startswith('ucast_'): x, y = map(int, name[len('ucast_'):].split('_')) child, next_idx = convert(next_idx) if y >= x: return _zext(solver, child, y), next_idx return _clip_unsigned_bv(solver, child, y), next_idx </pre>	cast the input unsigned integer with src_bits to dst_bits, if src_bits > dst_bits, then round the input to the nearest integer, and clip to the range of unsigned integer with dst_bits condition: $x \neq y$; $x,y \in \{1..14\}$; 182 pairs total

primitive	signature	Verilog translation & example output	CVC5 translation	comment
ulshift_x_byN	UInt(x) → UInt(x+N)	<pre> if name.startswith('ulshift_'): rest = name[len('ulshift_'):] x_str, k_str = rest.split('_by') x, k = int(x_str), int(k_str) out = x + k ce, cw, next_idx = convert(next_idx) ext = _zext(ce, cw, out, emit_forced) t, tw = emit(out, f"{ext} << {k}") return t, tw, next_idx //ulshift_8_by2 wire [9:0] t0 = {{2{1'b0}}}, ma} << 2; </pre>	<pre> if name.startswith('ulshift_'): rest = name[len('ulshift_'):] x_str, k_str = rest.split('_by') x, k = int(x_str), int(k_str) out = x + k child, next_idx = convert(next_idx) child_ext = _zext(solver, child, out) shift_amount = solver.mkBitVector(out, k) return solver.mkTerm(Kind.BITVECTOR_SHL, child_ext, shift_amount), next_idx </pre>	left shift the src_bits for amount bit condition: $x+N \leq 14$; $x \in \{1..13\}$; 91 pairs total
ushr_x_y	UInt(x),UInt(y) → UInt(x)	<pre> if name.startswith('ushr_'): x, y = map(int, name[len('ushr_'):].split('_')) ve, vw, next_idx = convert(next_idx) ae, aw, next_idx = convert(next_idx) t, tw = emit(x, f"{ve} >> {ae}") return t, tw, next_idx //ushr_8_3 wire [2:0] t0 = 3'd0; wire [7:0] t1 = ma >> t0; </pre>	<pre> if name.startswith('ushr_'): x, y = map(int, name[len('ushr_'):].split('_')) value, next_idx = convert(next_idx) amount, next_idx = convert(next_idx) return _ushr_bv(solver, value, amount, x), next_idx </pre>	input 2 unsigned integer with x_bits and y_bits, output a unsigned integer with x_bits, the output is the result of $a \gg b$, if $b \geq x_bits$, then the output is 0 no restriction; $x,y \in \{1..14\}$; 196 pairs total
ushl_x_y	UInt(x),UInt(y) → UInt(x)	<pre> if name.startswith('ushl_'): x, y = map(int, name[len('ushl_'):].split('_')) ve, vw, next_idx = convert(next_idx) ae, aw, next_idx = convert(next_idx) wide = x + y v_wide = _zext(ve, vw, wide, emit_forced) shifted, _ = emit(wide, f"{v_wide} << {ae}") t, tw = _clip_unsigned_verilog(emit, emit_forced, shifted, wide, x) return t, tw, next_idx //ushl_8_3 wire [2:0] t0 = 3'd0; wire [10:0] t1 = {{3{1'b0}}}, ma} << t0; wire [10:0] t2 = ((t1 > 11'd255)) ? 11'd255 : t1; wire [7:0] t3 = t2[7:0]; </pre>	<pre> if name.startswith('ushl_'): x, y = map(int, name[len('ushl_'):].split('_')) value, next_idx = convert(next_idx) amount, next_idx = convert(next_idx) return _ushl_bv(solver, value, amount, x), next_idx </pre>	same as above but left shift no restriction; $x,y \in \{1..14\}$; 196 pairs total
sticky_x_y	UInt(x),UInt(y) → Sign	<pre> if name.startswith('sticky_'): x, y = map(int, name[len('sticky_'):].split('_')) ve, vw, next_idx = convert(next_idx) ae, aw, next_idx = convert(next_idx) shr, _ = emit(x, f"{ve} >> {ae}") shl_back, _ = emit(x, f"{shr} << {ae}") t, tw = emit(1, f"({shl_back} != {ve})") return t, tw, next_idx //sticky_8_3 wire [2:0] t0 = 3'd0; wire [7:0] t1 = ma >> t0; wire [7:0] t2 = t1 << t0; wire [0:0] t3 = (t2 != ma); </pre>	<pre> if name.startswith('sticky_'): x, y = map(int, name[len('sticky_'):].split('_')) value, next_idx = convert(next_idx) amount, next_idx = convert(next_idx) return _sticky_bv(solver, value, amount, x), next_idx </pre>	the sign just show wheter the stick is all 0 no restriction; $x,y \in \{1..14\}$; 196 pairs total
msb_ux	UInt(x) → Sign	<pre> if name.startswith('msb_u'): n = int(name[len('msb_u'):]) ce, cw, next_idx = convert(next_idx) ce, _ = emit_forced(cw, ce) t, tw = emit(1, f"{ce}[{n - 1}]") return t, tw, next_idx //msb_u4 wire [3:0] t0 = 4'd10; wire [3:0] t1 = t0; wire [0:0] t2 = t1[3]; </pre>	<pre> if name.startswith('msb_u'): n = int(name[len('msb_u'):]) child, next_idx = convert(next_idx) return solver.mkTerm(solver.mkOp(Kind.BITVECTOR_EXTRACT, n - 1, n - 1), child), next_idx </pre>	get the top bit of an x-bit unsigned integer, as a Sign single parameter; $x \in \{1..14\}$; 14 instances total

primitive	signature	Verilog translation & example output	CVC5 translation	comment
ugt_ux	UInt(x), UInt(x) → Sign	<pre> if name.startswith('ugt_u'): n = int(name[len('ugt_u'):]) le, lw, next_idx = convert(next_idx) re, rw, next_idx = convert(next_idx) t, tw = emit(1, f"({le} > {re})") return t, tw, next_idx //ugt_u4 wire [3:0] t0 = 4'd10; wire [3:0] t1 = 4'd3; wire [0:0] t2 = (t0 > t1); </pre>	<pre> if name.startswith('ugt_u'): left, next_idx = convert(next_idx) right, next_idx = convert(next_idx) gt_bool = solver.mkTerm(Kind.BITVECTOR_UGT, left, right) return solver.mkTerm(Kind.ITE, gt_bool, solver.mkBitVector(1, 1), solver.mkBitVector(1, 0)), next_idx </pre>	compare two unsigned integers single parameter; x ∈ {1..14}; 14 instances total
ite_ux	Sign, UInt(x), UInt(x) → UInt(x)	<pre> if name.startswith('ite_u'): n = int(name[len('ite_u'):]) cnde, cndw, next_idx = convert(next_idx) ae, aw, next_idx = convert(next_idx) be, bw, next_idx = convert(next_idx) t, tw = emit(n, f"({cnde}) ? {ae} : {be}") return t, tw, next_idx //ite_u4 wire [0:0] t0 = 1'b0; wire [3:0] t1 = 4'd10; wire [3:0] t2 = 4'd3; wire [3:0] t3 = (t0) ? t1 : t2; </pre>	<pre> if name.startswith('ite_u'): n = int(name[len('ite_u'):]) cond, next_idx = convert(next_idx) a, next_idx = convert(next_idx) b, next_idx = convert(next_idx) cond_bool = solver.mkTerm(Kind.EQUAL, cond, solver.mkBitVector(1, 1)) return solver.mkTerm(Kind.ITE, cond_bool, a, b), next_idx </pre>	if cond is True return a else return b, both a and b are x-bit unsigned integer single parameter; x ∈ {1..14}; 14 instances total
uzero_x / uone_x	constant UInt(x)	<pre> if name.startswith('uzero_'): w = int(name[len('uzero_'):]) t, tw = emit(w, f"{w}'d0") return t, tw, idx + 1 if name.startswith('uone_'): w = int(name[len('uone_'):]) t, tw = emit(w, f"{w}'d1") return t, tw, idx + 1 //uzero_4 wire [3:0] t0 = 4'd0; </pre>	<pre> if name.startswith('uzero_'): w = int(name[len('uzero_'):]) return solver.mkBitVector(w, 0), idx + 1 if name.startswith('uone_'): w = int(name[len('uone_'):]) return solver.mkBitVector(w, 1), idx + 1 </pre>	terminal terminal, single parameter; x ∈ {1..14} each; 14 each
xor_sign	Sign, Sign → Sign	<pre> if name == 'xor_sign': le, lw, next_idx = convert(next_idx) re, rw, next_idx = convert(next_idx) t, tw = emit(1, f"({le} ^ {re})") return t, tw, next_idx //xor_sign wire [0:0] t0 = 1'b0; wire [0:0] t1 = 1'b1; wire [0:0] t2 = t0 ^ t1; </pre>	<pre> if name == 'xor_sign': left, next_idx = convert(next_idx) right, next_idx = convert(next_idx) return solver.mkTerm(Kind.BITVECTOR_XOR, left, right), next_idx </pre>	xor the input 2 sign, and return the result as a sign no parameter; 1 instance only
ite_sign	Sign, Sign, Sign → Sign	<pre> if name == 'ite_sign': cnde, cndw, next_idx = convert(next_idx) ae, aw, next_idx = convert(next_idx) be, bw, next_idx = convert(next_idx) t, tw = emit(1, f"({cnde}) ? {ae} : {be}") return t, tw, next_idx //ite_sign wire [0:0] t0 = 1'b0; wire [0:0] t1 = 1'b1; wire [0:0] t2 = (t0) ? t0 : t1; </pre>	<pre> if name == 'ite_sign': cond, next_idx = convert(next_idx) a, next_idx = convert(next_idx) b, next_idx = convert(next_idx) cond_bool = solver.mkTerm(Kind.EQUAL, cond, solver.mkBitVector(1, 1)) return solver.mkTerm(Kind.ITE, cond_bool, a, b), next_idx </pre>	if then else no parameter; 1 instance only
bit_to_u1	Sign → UInt(1)	<pre> if name == 'bit_to_u1': ce, cw, next_idx = convert(next_idx) t, tw = emit(1, ce) return t, tw, next_idx //bit_to_u1 wire [0:0] t0 = 1'b0; wire [0:0] t1 = t0; </pre>	<pre> if name == 'bit_to_u1': child, next_idx = convert(next_idx) return child, next_idx </pre>	translate the carry or not to +1 or not no parameter; 1 instance only

primitive	signature	Verilog translation & example output	CVC5 translation	comment
positive_sign / negative_sign	constant Sign	<pre> if name == 'positive_sign': t, tw = emit(1, "1'b0") return t, tw, idx + 1 if name == 'negative_sign': t, tw = emit(1, "1'b1") return t, tw, idx + 1 //positive_sign wire [0:0] t0 = 1'b0; </pre>	<pre> if name == 'positive_sign': return solver.mkBitVector(1, 0), idx + 1 if name == 'negative_sign': return solver.mkBitVector(1, 1), idx + 1 </pre>	terminal terminal, no parameter; 1 each
sign_e{e}m{m}	UInt(in_width) → Sign	<pre> if name.startswith('sign_e'): ce, cw, next_idx = convert(next_idx) t, tw = emit(1, f"{{ce}}[{{e + m}}]") return t, tw, next_idx //sign_e4m3 wire [0:0] t0 = ma[7]; </pre>	<pre> if name.startswith('sign_e'): child, next_idx = convert(next_idx) return solver.mkTerm(solver.mkOp(Kind.BITVECTOR_EXTRACT, e + m, e + m), child), next_idx </pre>	make the sign primitives, the input is the unsigned integer with 8(in_width) bits, the output is the sign bit(e.g. 0 for neg, q for pos) condition: $\max(x,y) \leq 13$; $x,y \in \{1..13\}$; 169 pairs total0
expf_e{e}m{m}	UInt(in_width) → UInt(e)	<pre> if name.startswith('expf_e'): ce, cw, next_idx = convert(next_idx) t, tw = emit(e, f"{{ce}}[{{e + m - 1}}:{{m}}]") return t, tw, next_idx //expf_e4m3 wire [3:0] t0 = ma[6:3]; </pre>	<pre> if name.startswith('expf_e'): child, next_idx = convert(next_idx) return solver.mkTerm(solver.mkOp(Kind.BITVECTOR_EXTRACT, e + m - 1, m), child), next_idx </pre>	make the exponent primitives, the input is the unsigned integer with 8, the output is the exponent with e bits condition: $\max(x,y) \leq 13$; $x,y \in \{1..13\}$; 169 pairs total0
sig_e{e}m{m}	UInt(in_width) → UInt(m+1)	<pre> if name.startswith('sig_e'): ce, cw, next_idx = convert(next_idx) exp_field = f"{{ce}}[{{e + m - 1}}:{{m}}]" frac_field = f"{{ce}}[{{m - 1}}:0]" top_bit = f"{{ce}}[{{m - 1}}]" if m >= 2: is_ext_subnormal = f"(({{exp_field}} == {{e}}'d0) && {{top_bit}})" shifted = f"{{" + f"{{ce}}[{{m - 2}}:0], 1'b0" + "}" sig_expr = f"({{is_ext_subnormal}} ? {{1'b1, {{shifted}}} : {{1'b1, {{frac_field}}}})" else: is_ext_subnormal = f"(({{exp_field}} == {{e}}'d0) && {{top_bit}})" sig_expr = f"({{is_ext_subnormal}} ? {{1'b1, 1'b0}} : {{1'b1, {{frac_field}}}})" t, tw = emit(m + 1, sig_expr) return t, tw, next_idx //sig_e4m3 wire [3:0] t0 = (((ma[6:3] == 4'd0) && ma[2])) ? {1'b1, {ma[1:0], 1'b0}} : {1'b1, ma[2:0]}; </pre>	<pre> if name.startswith('sig_e'): child, next_idx = convert(next_idx) frac = solver.mkTerm(solver.mkOp(Kind.BITVECTOR_EXTRACT, m - 1, 0), child) hidden = solver.mkBitVector(1, 1) return solver.mkTerm(Kind.BITVECTOR_CONCAT, hidden, frac), next_idx </pre>	the input is the unsigned integer with 8, the output is the significand with m+1 bits(plus the 1 in 1.xxx) condition: $\max(x,y) \leq 13$; $x,y \in \{1..13\}$; 169 pairs total0
encode_e{eo}m{mo}	Sign, UInt(eo), UInt(mo) → UInt(out_width)	<pre> if name.startswith('encode_e'): se, sw, next_idx = convert(next_idx) ee, ew, next_idx = convert(next_idx) fe, fw, next_idx = convert(next_idx) t, tw = emit(out_width, f"{{" + f"{{se}}, {{ee}}, {{fe}}" + "}})" return t, tw, next_idx //encode_e5m4 wire [0:0] t0 = 1'b0; wire [4:0] t1 = 5'd0; wire [3:0] t2 = 4'd0; wire [9:0] t3 = {t0, t1, t2}; </pre>	<pre> if name.startswith('encode_e'): sign, next_idx = convert(next_idx) expf, next_idx = convert(next_idx) frac, next_idx = convert(next_idx) return solver.mkTerm(Kind.BITVECTOR_CONCAT, solver.mkTerm(Kind.BITVECTOR_CONCAT, sign, expf), frac), next_idx </pre>	make the whole minifloat number with the three parts condition: $\max(x,y) \leq 13$; $x,y \in \{1..13\}$; 169 pairs total1

primitive	signature	Verilog translation & example output	CVC5 translation	comment
uconcat_x_y	UInt(x),UInt(y) → UInt(x+y)	<pre> if name.startswith('uconcat_'): x, y = map(int, name[len('uconcat_')].split('_')) out = x + y le, lw, next_idx = convert(next_idx) re, rw, next_idx = convert(next_idx) t, tw = emit(out, "{" + f"{le}, {re}" + "}") return t, tw, next_idx //uconcat_4_4 wire [3:0] t0 = 4'd10; wire [3:0] t1 = 4'd3; wire [7:0] t2 = {t0, t1}; </pre>	<pre> if name.startswith('uconcat_'): left, next_idx = convert(next_idx) right, next_idx = convert(next_idx) return solver.mkTerm(Kind.BITVECTOR_CONCAT, left, right), next_idx </pre>	concat x and y together condition: $\max(x,y) \leq 13$; $x,y \in \{1..13\}$; 169 pairs total2
umul_x_y	UInt(x),UInt(y) → UInt(x+y)	<pre> if name.startswith('umul_'): x, y = map(int, name[len('umul_')].split('_')) out = x + y le, lw, next_idx = convert(next_idx) re, rw, next_idx = convert(next_idx) l = _zext(le, lw, out, emit_forced) r = _zext(re, rw, out, emit_forced) t, tw = emit(out, f"{l} * {r}") return t, tw, next_idx //umul_4_4 wire [3:0] t0 = 4'd10; wire [3:0] t1 = 4'd3; wire [7:0] t2 = {4{1'b0}}, t0} * {4{1'b0}}, t1}; </pre>	<pre> if name.startswith('umul_'): x, y = map(int, name[len('umul_')].split('_')) out = x + y left, next_idx = convert(next_idx) right, next_idx = convert(next_idx) l = _zext(solver, left, out) r = _zext(solver, right, out) return solver.mkTerm(Kind.BITVECTOR_MULT, l, r), next_idx </pre>	unsigned mult, add, condition: $\max(x,y) \leq 13$; $x,y \in \{1..13\}$; 169 pairs total2
sigmul_x_y	UInt(x),UInt(y) → UInt(x+y)	<pre> if name.startswith('sigmul_'): x, y = map(int, name[len('sigmul_')].split('_')) out = x + y le, lw, next_idx = convert(next_idx) re, rw, next_idx = convert(next_idx) memo_key = (name, le, re) if memo_key in submodule_call_memo: cached_t, cached_tw = submodule_call_memo[memo_key] return cached_t, cached_tw, next_idx needed_sigmul_modules.add((x, y)) mod_name = f"sigmul_{x}_{y}" t, tw = emit_instance(mod_name, {"X": le, "Y": re}, "R", out) submodule_call_memo[memo_key] = (t, tw) return t, tw, next_idx //sigmul_4_4 module sigmul_4_4(input [3:0] X, input [3:0] Y, output [7:0] R); assign R = X * Y; endmodule wire [3:0] t0 = ((ma[6:3] == 4'd0) && ma[2])) ? {1'b1, {ma[1:0], 1'b0}} : {1'b1, ma[2:0]}; wire [3:0] t1 = ((mb[6:3] == 4'd0) && mb[2])) ? {1'b1, {mb[1:0], 1'b0}} : {1'b1, mb[2:0]}; wire [7:0] t2; sigmul_4_4 sigmul_4_4_inst_1 (.X(t0), .Y(t1), .R(t2)); </pre>	<pre> if name.startswith('sigmul_'): x, y = map(int, name[len('sigmul_')].split('_')) out = x + y left, next_idx = convert(next_idx) right, next_idx = convert(next_idx) l = _zext(solver, left, out) r = _zext(solver, right, out) return solver.mkTerm(Kind.BITVECTOR_MULT, l, r), next_idx </pre>	though these two looks the same, the difference is in the synth file, one is wire = l * r, one is make a module called sigmul4_4 and out it in it, not in the top module in yosys condition: $\max(x,y) \leq 13$; $x,y \in \{1..13\}$; 169 pairs total2
usubwrap_x_y	UInt(x),UInt(y) → UInt(max(x,y)+1)	<pre> if name.startswith('usubwrap_'): x, y = map(int, name[len('usubwrap_')].split('_')) out = max(x, y) + 1 le, lw, next_idx = convert(next_idx) re, rw, next_idx = convert(next_idx) l = _zext(le, lw, out, emit_forced) r = _zext(re, rw, out, emit_forced) t, tw = emit(out, f"{l} - {r}") return t, tw, next_idx //usubwrap_8_6 wire [5:0] t0 = 6'd0; wire [8:0] t1 = {{1{1'b0}}, ma} - {{3{1'b0}}, t0}; </pre>	<pre> if name.startswith('usubwrap_'): x, y = map(int, name[len('usubwrap_')].split('_')) out = max(x, y) + 1 left, next_idx = convert(next_idx) right, next_idx = convert(next_idx) l = _zext(solver, left, out) r = _zext(solver, right, out) return solver.mkTerm(Kind.BITVECTOR_SUB, l, r), next_idx </pre>	these two looks the same too, but their difference is in whether the output is floored at 0 or wrapped around condition: $\max(x,y) \leq 13$; $x,y \in \{1..13\}$; 169 pairs total

primitive	signature	Verilog translation & example output	CVC5 translation	comment
roundadd_x_y	UInt(x),UInt(y), Sign $\rightarrow$ UInt(max(x, y))	<pre> if name.startswith('roundadd_'): x, y = map(int, name[len('roundadd_')].split('_')) out = max(x, y) le, lw, next_idx = convert(next_idx) re, rw, next_idx = convert(next_idx) ce, cw, next_idx = convert(next_idx) memo_key = (name, le, re, ce) if memo_key in submodule_call_memo: cached_t, cached_tw = submodule_call_memo[memo_key] return cached_t, cached_tw, next_idx needed_roundadd_modules.add((x, y)) mod_name = f"roundadd_{x}_{y}" t, tw = emit_instance(mod_name, {"X": le, "Y": re, "Cin": ce}, "R", out) submodule_call_memo[memo_key] = (t, tw) return t, tw, next_idx //roundadd_4_4 module roundadd_4_4(input [3:0] X, input [3:0] Y, input Cin, output [3:0] R); assign R = X + Y + Cin; endmodule wire [3:0] t0 = 4'd0; wire [0:0] t1 = 1'b0; wire [3:0] t2; roundadd_4_4 roundadd_4_4_inst_1 (.X(t0), .Y(t0), .Cin(t1), .R(t2)); </pre>	<pre> if name.startswith('roundadd_'): x, y = map(int, name[len('roundadd_')].split('_')) out = max(x, y) left, next_idx = convert(next_idx) right, next_idx = convert(next_idx) cin, next_idx = convert(next_idx) l = _zext(solver, left, out) r = _zext(solver, right, out) c = _zext(solver, cin, out) sum1 = solver.mkTerm(Kind.BITVECTOR_ADD, l, r) return solver.mkTerm(Kind.BITVECTOR_ADD, sum1, c), next_idx </pre>	this is the round add, which is used in the rounding process, the input is 2 unsigned integer with x and y bits, and a sign bit, the output is an unsigned integer with max(x,y) bits, which is the sum of the 2 unsigned integer and the sign bit(if it is True, then add 1 to the sum), as it's wraparound add, it dont work on the extend bit caused by carry condition: $\max(x,y) \leq 13$; $x,y \in \{1..13\}$; 169 pairs total
classify_e{e}m{m}	UInt(in_width) $\rightarrow$ UInt(2)	<pre> if name.startswith('classify_e'): rest = name[len('classify_e'):] e_str, m_str = rest.split('m') e_val, m_val = int(e_str), int(m_str) ce, cw, next_idx = convert(next_idx) if split_classify: mod_name = f"classify_e{e_val}m{m_val}" needed_classify_modules.add((e_val, m_val)) t, tw = emit_instance(mod_name, {"bits": ce}, "tag", 2) return t, tw, next_idx ce, _ = emit_forced(cw, ce) max_exp = (1 << e_val) - 1 exp_field = f"{ce}[{e_val + m_val - 1}:{m_val}]" frac_field = f"{ce}[{m_val - 1}:0]" is_exp_zero = f"({exp_field} == {e_val}'d0)" is_exp_infty = f"({exp_field} == {e_val}'d{max_exp})" is_frac_zero = f"({frac_field} == {m_val}'d0)" is_repr_subnormal = f"{ce}[{m_val - 1}]" is_zero = f"({is_exp_zero} && !({is_repr_subnormal}))" is_infinity = f"({is_exp_infty} && {is_frac_zero})" is_nan = f"({is_exp_infty} && !({is_frac_zero}))" expr = f"({is_zero} ? 2'd0 : ({is_infinity} ? 2'd2 : ({is_nan} ? 2'd3 : 2'd1)))" t, tw = emit(2, expr) return t, tw, next_idx //classify_e4m3 module classify_e4m3(input [7:0] bits, output [1:0] tag); wire [3:0] expf = bits[6:3]; wire [2:0] frac = bits[2:0]; wire is_exp_zero = (expf == 4'd0); wire is_exp_infty = (expf == 4'd15); wire is_frac_zero = (frac == 3'd0); wire is_repr_subnormal = frac[2]; wire is_zero = is_exp_zero && !is_repr_subnormal; wire is_infinity = is_exp_infty && is_frac_zero; wire is_nan = is_exp_infty && !is_frac_zero; assign tag = is_zero ? 2'd0 : (is_infinity ? 2'd2 : (is_nan ? 2'd3 : 2'd1)); endmodule wire [1:0] t0; classify_e4m3 classify_e4m3_inst_1 (.bits(ma), .tag(t0)); </pre>	<pre> if name.startswith('classify_e'): rest = name[len('classify_e'):] e_val_str, m_val_str = rest.split('m') e_val, m_val = int(e_val_str), int(m_val_str) child, next_idx = convert(next_idx) exp_mask_val = (1 << e_val) - 1 expf = solver.mkTerm(solver.mkOp(Kind.BITVECTOR_EXTRACT, e_val + m_val - 1, m_val), child) frac = solver.mkTerm(solver.mkOp(Kind.BITVECTOR_EXTRACT, m_val - 1, 0), child) exp_zero = solver.mkTerm(Kind.EQUAL, expf, solver.mkBitVector(e_val, 0)) exp_infty = solver.mkTerm(Kind.EQUAL, expf, solver.mkBitVector(e_val, exp_mask_val)) frac_zero = solver.mkTerm(Kind.EQUAL, frac, solver.mkBitVector(m_val, 0)) repr_subnormal_bit = solver.mkTerm(solver.mkOp(Kind.BITVECTOR_EXTRACT, m_val - 1, m_val - 1), frac) repr_subnormal = solver.mkTerm(Kind.EQUAL, repr_subnormal_bit, solver.mkBitVector(1, 1)) infinity = solver.mkTerm(Kind.AND, exp_infty, frac_zero) not_repr_subnormal = solver.mkTerm(Kind.NOT, repr_subnormal) zero_flag = solver.mkTerm(Kind.AND, exp_zero, not_repr_subnormal) not_frac_zero = solver.mkTerm(Kind.NOT, frac_zero) nan_flag = solver.mkTerm(Kind.AND, exp_infty, not_frac_zero) result = solver.mkTerm(Kind.ITE, zero_flag, solver.mkBitVector(2, 0), solver.mkTerm(Kind.ITE, infinity, solver.mkBitVector(2, 2), solver.mkTerm(Kind.ITE, nan_flag, solver.mkBitVector(2, 3), solver.mkBitVector(2, 1)))) return result, next_idx </pre>	make the classify of exception tag(abnormal or not) primitives, the input is the unsigned integer with 8, the output is the exception tag with 2 bits(00,01,11,10) condition: $\max(x,y) \leq 13$; $x,y \in \{1..13\}$; 169 pairs total0

primitive	signature	Verilog translation & example output	CVC5 translation	comment
input_ieee_e{e}m{m}	UInt(in_width) → UInt(2+in_width)	<pre> if name.startswith('input_ieee_e'): rest = name[len('input_ieee_e'):] e_val_str, m_val_str = rest.split('m') e_val, m_val = int(e_val_str), int(m_val_str) ce, cw, next_idx = convert(next_idx) memo_key = (name, ce) if memo_key in submodule_call_memo: cached_t, cached_tw = submodule_call_memo[memo_key] return cached_t, cached_tw, next_idx mod_name = f"input_ieee_e{e_val}m{m_val}" needed_input_ieee_modules.add((e_val, m_val)) out_w = 2 + e_val + m_val + 1 t, tw = emit_instance(mod_name, {"bits": ce}, "out", out_w) submodule_call_memo[memo_key] = (t, tw) return t, tw, next_idx //input_ieee_e4m3 module input_ieee_e4m3(input [7:0] bits, output [9:0] out); wire [3:0] expf = bits[6:3]; wire [2:0] frac = bits[2:0]; wire sign = bits[7]; wire is_exp_zero = (expf == 4'd0); wire is_exp_infty = (expf == 4'd15); wire is_frac_zero = (frac == 3'd0); wire is_repr_subnormal = frac[2]; wire [2:0] sfrac = (is_exp_zero && is_repr_subnormal) ? {frac[1:0], 1'b0} : frac; wire is_zero = is_exp_zero && !is_repr_subnormal; wire is_infinity = is_exp_infty && is_frac_zero; wire is_nan = is_exp_infty && !is_frac_zero; wire [1:0] tag = is_zero ? 2'd0 : (is_infinity ? 2'd2 : (is_nan ? 2'd3 : 2'd1)); assign out = {tag, sign, expf, sfrac}; endmodule wire [9:0] t0; input_ieee_e4m3 input_ieee_e4m3_inst_1 (.bits(ma), .out(t0)); </pre>	<pre> if name.startswith('input_ieee_e'): rest = name[len('input_ieee_e'):] e_val_str, m_val_str = rest.split('m') e_val, m_val = int(e_val_str), int(m_val_str) child, next_idx = convert(next_idx) exp_mask_val = (1 << e_val) - 1 sign = solver.mkTerm(solver.mkOp(Kind.BITVECTOR_EXTRACT, e_val + m_val, e_val + m_val), child) expf = solver.mkTerm(solver.mkOp(Kind.BITVECTOR_EXTRACT, e_val + m_val - 1, m_val), child) frac = solver.mkTerm(solver.mkOp(Kind.BITVECTOR_EXTRACT, m_val - 1, 0), child) exp_zero = solver.mkTerm(Kind.EQUAL, expf, solver.mkBitVector(e_val, 0)) exp_infty = solver.mkTerm(Kind.EQUAL, expf, solver.mkBitVector(e_val, exp_mask_val)) frac_zero = solver.mkTerm(Kind.EQUAL, frac, solver.mkBitVector(m_val, 0)) repr_subnormal_bit = solver.mkTerm(solver.mkOp(Kind.BITVECTOR_EXTRACT, m_val - 1, m_val - 1), frac) repr_subnormal = solver.mkTerm(Kind.EQUAL, repr_subnormal_bit, solver.mkBitVector(1, 1)) if m_val > 1: frac_low = solver.mkTerm(solver.mkOp(Kind.BITVECTOR_EXTRACT, m_val - 2, 0), frac) sfrac_sub = solver.mkTerm(Kind.BITVECTOR_CONCAT, frac_low, solver.mkBitVector(1, 0)) else: sfrac_sub = solver.mkBitVector(m_val, 0) exp_zero_and_subnorm = solver.mkTerm(Kind.AND, exp_zero, repr_subnormal) sfrac = solver.mkTerm(Kind.ITE, exp_zero_and_subnorm, sfrac_sub, frac) infinity = solver.mkTerm(Kind.AND, exp_infty, frac_zero) not_repr_subnormal = solver.mkTerm(Kind.NOT, repr_subnormal) zero_flag = solver.mkTerm(Kind.AND, exp_zero, not_repr_subnormal) not_frac_zero = solver.mkTerm(Kind.NOT, frac_zero) nan_flag = solver.mkTerm(Kind.AND, exp_infty, not_frac_zero) exn = solver.mkTerm(Kind.ITE, zero_flag, solver.mkBitVector(2, 0), solver.mkTerm(Kind.ITE, infinity, solver.mkBitVector(2, 2), solver.mkTerm(Kind.ITE, nan_flag, solver.mkBitVector(2, 3), solver.mkBitVector(2, 1)))) packed = solver.mkTerm(Kind.BITVECTOR_CONCAT, exn, solver.mkTerm(Kind.BITVECTOR_CONCAT, sign, solver.mkTerm(Kind.BITVECTOR_CONCAT, expf, sfrac))) return packed, next_idx </pre>	the input is the unsigned integer with 8, the output is 2+8 bits with the minifloat_8 and the exception tag condition: $\max(x,y) \leq 13$; $x,y \in \{1..13\}$; 169 pairs total

primitive	signature	Verilog translation & example output	CVC5 translation	comment
output_ieee_e{eo} m{mo}	UInt(2),Sign, UInt(eo),UInt(mo) → UInt(out_width)	<pre> if name.startswith('output_ieee_e'): rest = name[len('output_ieee_e'):] e_val_str, m_val_str = rest.split('m') eo_val, mo_val = int(e_val_str), int(m_val_str) xe, xw, next_idx = convert(next_idx) se, sw, next_idx = convert(next_idx) ee, ew, next_idx = convert(next_idx) fe, fw, next_idx = convert(next_idx) xe, _ = emit_forced(xw, xe) ee, _ = emit_forced(ew, ee) fe, _ = emit_forced(fw, fe) exp_zero = f"({ee} == {eo_val})'d0" out_sign = f"(({xe}==2'd0 {xe}==2'd1 {xe}==2'd2) ? {se} : 1'b0)" if mo_val >= 2: boundary_frac = "{" + f"1'b1, {fe}[{mo_val - 1}:1]" + "}" else: boundary_frac = "1'b1" frac_r = (f"({xe}==2'd0) ? {mo_val}'d0 : " f"(({exp_zero}) && {xe}==2'd1) ? {boundary_frac} : " f"({xe}==2'd1) ? {fe} : " "{" + f"{{{mo_val - 1}}{1'b0}}}", {xe}[0]" + "}") exp_r = (f"({xe}==2'd0) ? {eo_val}'d0 : " f"({xe}==2'd1) ? {ee} : " "{" + f"{eo_val}{{1'b1}}}" + "}") t, tw = emit(out_width, "{" + f"{out_sign}, ({exp_r}), ({frac_r})" + "}") return t, tw, next_idx //output_ieee_e5m4 wire [1:0] t0 = 2'd0; wire [0:0] t1 = 1'b0; wire [4:0] t2 = 5'd0; wire [3:0] t3 = 4'd0; wire [1:0] t4 = t0; wire [4:0] t5 = t2; wire [3:0] t6 = t3; wire [9:0] t7 = {((t4==2'd0 t4==2'd1 t4==2'd2) ? t1 : 1'b0), ((t4==2'd0) ? 5'd0 : (t4==2'd1) ? t5 : {5{1'b1}}), ((t4==2'd0) ? 4'd0 : (((t5 == 5'd0)) && t4==2'd1) ? {1'b1, t6[3:1]} : (t4==2'd1) ? t6 : {{3{1'b0}}, t4[0]}}); </pre>	<pre> if name.startswith('output_ieee_e'): rest = name[len('output_ieee_e'):] eo_str, mo_str = rest.split('m') eo_val, mo_val = int(eo_str), int(mo_str) exn, next_idx = convert(next_idx) sign, next_idx = convert(next_idx) expf, next_idx = convert(next_idx) frac, next_idx = convert(next_idx) exp_mask_val = (1 << eo_val) - 1 exn_is0 = solver.mkTerm(Kind.EQUAL, exn, solver.mkBitVector(2, 0)) exn_is1 = solver.mkTerm(Kind.EQUAL, exn, solver.mkBitVector(2, 1)) exn_le2 = solver.mkTerm(Kind.BITVECTOR_ULE, exn, solver.mkBitVector(2, 2)) exp_zero = solver.mkTerm(Kind.EQUAL, expf, solver.mkBitVector(eo_val, 0)) out_sign = solver.mkTerm(Kind.ITE, exn_le2, sign, solver.mkBitVector(1, 0)) one_mo = solver.mkBitVector(mo_val, 1) frac_shr1 = solver.mkTerm(Kind.BITVECTOR_LSHR, frac, one_mo) hidden_bit_tag = solver.mkBitVector(mo_val, 1 << (mo_val - 1)) subnorm_frac = solver.mkTerm(Kind.BITVECTOR_OR, hidden_bit_tag, frac_shr1) exn_is1_and_expzero = solver.mkTerm(Kind.AND, exn_is1, exp_zero) exn_and_1 = _zext(solver, solver.mkTerm(solver.mkOp(Kind.BITVECTOR_EXTRACT, 0, 0), exn), mo_val) frac_r = solver.mkTerm(Kind.ITE, exn_is0, solver.mkBitVector(mo_val, 0), solver.mkTerm(Kind.ITE, exn_is1_and_expzero, subnorm_frac, solver.mkTerm(Kind.ITE, exn_is1, frac, exn_and_1))) exp_r = solver.mkTerm(Kind.ITE, exn_is0, solver.mkBitVector(eo_val, 0), solver.mkTerm(Kind.ITE, exn_is1, expf, solver.mkBitVector(eo_val, exp_mask_val))) packed = solver.mkTerm(Kind.BITVECTOR_CONCAT, out_sign, solver.mkTerm(Kind.BITVECTOR_CONCAT, exp_r, frac_r)) return packed, next_idx </pre>	the 10 bit with exception tag minifloat(flopoco format) condition: $\max(x,y) \leq 13$; $x,y \in \{1..13\}$; 169 pairs total

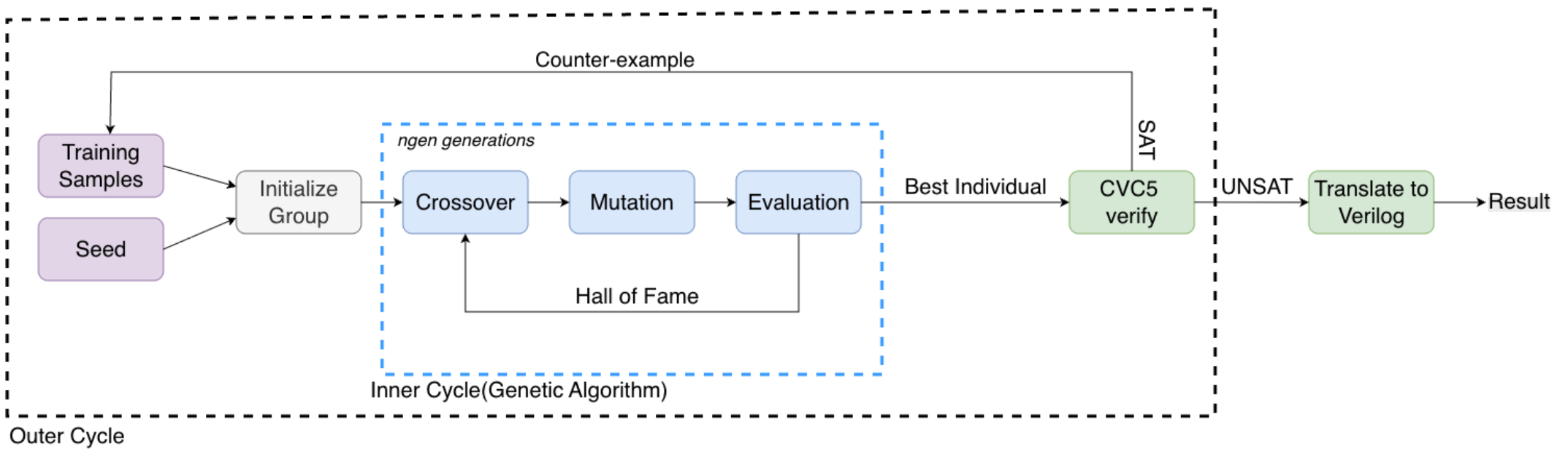


Figure 1: The whole pipeline